

[표준] 파이썬 코드 작성 규칙 및 Claude Code 자동 적용 절차

문서 목적 — 팀 공통 파이썬 코드 작성 규칙을 정의하고, 이 규칙이 사람과 Claude Code 양쪽에 동일하게 적용되도록 하는 배포·강제 절차를 규정한다. 본 문서는 사람이 읽는 단일 원본(Single Source of Truth)이며, 저장소의 설정 파일은 이 문서로부터 생성된 사본이다.

문서 식별자 — 라벨 `python-coding-standard`. 자동화는 페이지 ID가 아니라 이 라벨로 본 문서를 찾는다. 이 라벨은 사내에서 본 문서에만 부여한다.

규칙 버전 — `RULESET_VERSION = 14` (규칙 내용이 바뀔 때마다 1씩 올린다)

1. 적용 범위

- 표준 언어는 **파이썬**이며, 본 규칙은 사내 모든 파이썬 저장소에 적용한다.
- 사람이 직접 작성한 코드와 Claude Code가 생성한 코드에 **동일하게** 적용한다.
- 외부 라이브러리 벤더링 코드, 자동 생성 코드(protobuf 등), 마이그레이션 스크립트는 예외로 한다. 예외 경로는 §8.1의 `per-file-ignores`, §8.3의 pre-commit `exclude`, §8.5의 게이트, §8.7의 훅 스크립트에 **함께** 반영한다. 한 곳만 고치면 나머지 경로에서 예외가 동작하지 않는다.
- git submodule** 과 가상환경은 도구마다 기본 동작이 달라 별도로 다룬다. ruff는 gitignore된 경로(`.venv` 등)를 자동 제외하지만 submodule은 워킹트리에 있으므로 그대로 검사하고, pydoclint는 경로를 직접 받으므로 둘 다 검사한다. §8.5의 `check_naming.py` 는 `git ls-files` 기반이라 submodule을 보지 않는다. 세 검사의 대상 범위가 서로 다르므로 §8.1과 §8.5에 각각 반영해야 한다.
- 테스트 코드**는 독스트링·타입 힌트·함수명 검사에서 제외한다. §8.10이 테스트에 `D · ANN · N802` 를 면제하는데, pydoclint가 검사하는 것이 정확히 앞의 둘이고 `check_naming.py` 가 보는 것이 셋째다. 한쪽만 면제하면 어느 이름을 써도 통과할 수 없는 코드가 생긴다(§7.1). 테스트의 이름은 계약이 아니라 그 자리의 설명이며, 특히 **대역(double)** 클래스는 **홍내** 내는 라이브러리가 이름을 정한다 — `model_dump · query_points · ping` 은 우리가 바꿀 수 있는 이름이 아니다.

2. 네이밍 규칙

핵심 원칙은 *****함수는 동작, 변수는 대상*****이다. 동사_목적어 형태는 동작을 수행하는 대상, 즉 함수-메서드에만 적용한다. 변수와 클래스에까지 동사를 강제하면 PEP 8 및 표준 린터와 충돌하고 가독성이 오히려 떨어진다.

대상	규칙	예시
함수 / 메서드	<code>{동사}_{목적어}[_{한정}]</code> , <code>snake_case</code>	<code>load_config_file</code> , <code>validate_user_input</code> ,

		<code>render_frame_batch</code>
변수	<code>{명사}[_{한정}]</code> , snake_case. 동사로 시작하지 않는다	<code>user_count</code> , <code>frame_index</code> , <code>output_path</code>
불리언 변수	<code>is_</code> / <code>has_</code> / <code>should_</code> 접두어	<code>is_valid</code> , <code>has_alpha_channel</code>
클래스	명사, PascalCase	<code>FrameLoader</code> , <code>PromptBuilder</code>
상수 / 모듈 전역	대문자 SNAKE_CASE	<code>MAX_RETRY_COUNT</code> , <code>DEFAULT_TIMEOUT_SEC</code>
지역 변수 / 파라미터	소문자 snake_case	<code>batch_size</code>
내부 전용	단일 밑줄 접두어	<code>_build_payload</code> , <code>_cache</code>
모듈 / 패키지	소문자 snake_case, 짧게	<code>frame_loader.py</code>

- **술어 함수**는 불리언 변수와 같은 규칙을 따른다. `is_` / `has_` / `should_` / `can_` 로 시작하면 목적어만 붙여도 된다(`is_expired`, `has_permission`). 술어에 동사를 강제하면 `check_is_expired` 같은 겹말이 나온다.
- **클래스 메서드**는 목적어를 생략할 수 있다. 클래스명이 목적어 역할을 하므로 `PROMPTS.get()` · `STATE.save_relation()` 처럼 동사 하나로 뜻이 통한다. `store.get_prompt()` 보다 `PROMPTS.get()` 이 맞는 파이썬이다. 동사를 허용 목록에서 고르는 규칙 자체는 그대로 적용된다.
- **외부 인터페이스**를 그대로 따라야 하는 이름은 예외다. 표준 라이브러리·프레임워크가 이름을 정하는 경우가 있다 — `logging.Logger` 를 감싸는 어댑터의 `info` / `debug` / `error`, 컨텍스트 매니저의 `close`, arq 워커 설정의 `redis_settings` 등. 이름을 바꾸면 드롭인 교체 계약이 깨지고, 대개 **예외 없이 조용히 기본값으로 떨어진다**. 이 경우 §11의 억제 주석에 사유를 남긴다. 웹 프레임워크의 라우트 핸들러와 pydantic 밸리데이터, pytest 픽스처는 데코레이터로 식별되므로 §8.2가 자동으로 면제한다.
- **변수 이름**은 뜻을 담는다. 타입을 접미·접두로 붙이지 않는다(`name_s · i_count`) — 타입은 타입 힌트가 말한다. 이름이 말해야 하는 것은 그 값이 **무엇인가**다.
- **한 글자 이름**은 관용이 확립된 자리에만 쓴다 — 수식의 좌표, 한 줄 컴프리헨션. 루프 변수도 무엇을 도는지 이름에 담는다: `for t in turns` 가 아니라 `for turn in turns` 다.
- **축약**은 저장소에서 이미 통용되는 것만 쓰고 새로 만들지 않는다(`idx · ctx · cfg`). 읽는 사람이 뜻을 되짚어야 하는 이름은 짧아서 얻는 것보다 잃는 것이 크다.

2.1. 권장 동사 목록

동사는 아래 목록에서 선택한다. 목록을 고정해야 자동 검사가 가능하다.

```
1 get set create build make load save read write parse render convert
2 validate check run execute fetch send update delete remove add apply
3 compute calculate extract filter sort merge split init register
4 resolve handle process train predict infer export ensure
5 list find iter count format describe normalize
6 start stop reset refresh retry rename prepare drop mark
7 call publish subscribe request poll probe purge
8 assign monitor manage track wait close open
9 plan scan store record evaluate embed promote
```

목록의 어떤 동사도 그 동작을 정확히 뜻하지 못하면 다른 동사를 써도 된다. 뜻이 어긋나는 동사를 억지로 골라 쓰면 이름이 동작을 설명하지 못하게 되는데, 그것이 목록을 지키는 것보다 나쁘다. 이 경우 해당 줄에 `# naming: allow <사유>` 를 남겨 예외임을 드러낸다. 같은 동사가 여러 곳에서 반복해 필요해지면 팀에 공통 목록 추가를 제안한다.

같은 개념에는 항상 같은 동사를 쓴다. `get_user` 와 `fetch_user` 가 한 저장소에 공존하지 않도록 한다. 원격 I/O는 `fetch` , 메모리·속성 접근은 `get` 으로 통일한다.

3. 스코프와 상수

- 모듈 전역에 두는 값은 상수만 허용하며 대문자로 표기한다. 가변 전역 상태는 금지한다.
- 가변 상태가 필요하면 클래스 속성이나 명시적으로 전달되는 컨텍스트 객체로 표현한다.
- 매직 넘버·매직 스트링은 모듈 상단 상수로 승격한다.
- `global` 키워드 사용은 금지한다. 예외는 시그널 핸들러가 프로세스 종료 플래그를 내리는 경우 하나다. 핸들러 시그니처가 고정되어 있어 `global` 없이 표현하기 어렵고, 클래스로 감싸는 대안은 이득 없이 복잡도만 올린다. 이 경우 억제 주석에 사유를 남긴다.

4. 함수·모듈 분리 기준

상황	조치
같은 로직이 한 파일 안에서 3회 이상 반복	함수로 추출
2회 반복이지만 로직이 5줄을 넘거나 수정 시 함께 바뀌어야 함	함수로 추출
함수가 2개 이상 파일에서 사용됨	공용 모듈로 분리 후 import
모듈이 400줄 초과 또는 책임이 둘 이상	파일 분리
함수가 50줄 초과 또는 순환 복잡도 10 초과	분해

"2회 반복 즉시 추출"을 기계적으로 적용하면 성급한 추상화를 유발한다. 판단 기준은 반복 횟수가 아니라 "*"한쪽을 고칠 때 다른 쪽도 반드시 같이 고쳐야 하는가"이다. 우연히 모양만 같은 코드는 합치지 않는다.

5. 독스트링

구글 스타일을 사용한다. 공개 모듈·클래스·함수에는 필수, 내부 전용(`_` 접두어) 함수에는 한 줄 요약만 있어도 된다.

```
1 def load_frame_batch(source_dir: Path, batch_size: int = 8) -> list[Frame]:
2     """디렉터리에서 프레임을 읽어 배치 단위로 반환한다.
3
4     Args:
5         source_dir: 프레임 이미지가 들어 있는 디렉터리 경로.
6         batch_size: 한 배치에 담을 프레임 수. 기본값 8.
7
8     Returns:
9         읽어들인 Frame 객체의 리스트. 파일이 없으면 빈 리스트.
10
11     Raises:
12         FileNotFoundError: source_dir가 존재하지 않는 경우.
13         ValueError: batch_size가 1 미만인 경우.
14     """
```

- 첫 줄은 명령형 한 문장 요약으로 끝맺는다.
- `Args`의 항목은 실제 시그니처와 일치해야 한다(자동 검사 대상). 한 항목이 두 줄을 넘으면 이어지는 줄을 더 깊이 들여쓴다 — 같은 깊이로 두면 콜론 없는 새 항목으로 읽혀 독스트링 전체가 파싱 실패한다(`DOC001`).
- 값을 돌려주는 함수에는 `Returns` (제너레이터는 `Yields`)를 적는다. 함수 이름을 되풀이하는 문장은 없는 것만 못하다 — 검사만 통과시키고 읽는 사람에게서는 아무것도 주지 않는다. 빈 값이 나올 수 있는지, `None` 이 언제 나오는지, 단위가 무엇인지처럼 호출자가 알아야 할 것을 적는다.
- `Raises`에는 원칙적으로 이 함수의 몸통이 직접 던지는 예외만 적는다. 호출한 함수가 던지는 것을 습관적으로 베껴 적으면 실제와 어긋난다.
예외 — 전파가 곧 계약인 경계 함수. 라우트 핸들러·공개 파사드처럼 호출자가 그 예외를 받는 것이 인터페이스의 일부인 자리에서는 전파되는 예외도 적는다. 호출자가 404·422·503을 언제 받는지는 그 자리에만 적을 수 있고, 지우면 어디에도 안 남는다.
이 항목은 자동 검사 대상이 아니다(§8.5가 `--skip-checking-raises`로 끈다). 도구는 전파를 볼 수 없어 정당한 문서화를 오탐으로 잡는다 — 정확성은 리뷰가 본다(§7).
- 타입은 타입 힌트로 표현하고 독스트링에 중복 기재하지 않는다. §8.5의 `pydoclint` 옵션이 이 규칙에 맞춰져 있다 — 기본값으로 두면 독스트링에 타입을 요구해 이 항목과 충돌한다.
- `__init__`에는 독스트링을 붙인다. 생성자가 무엇을 준비하는지, 인자에 무엇이 오는지는 클래스 요약과 별개의 정보다. 다만 `pydoclint`는 기본값에서 이것을 클래스 독스트링에 합치라고 요구하므로(`DOC301`), §8.5에서 `--allow-init-docstring=True`로 끈다. 이 옵션이 없으면 불여도(`pydoclint` `DOC301`) 떼어도(`ruff` `D107`) 검사가 실패해 어느 쪽도 통과할 수 없다.

- 공개 API에는 타입 힌트를 필수로 붙인다. 애너테이션은 실제 타입과 일치해야 한다 — 게이트는 존재 여부만 보므로(\$7) 틀린 애너테이션도 통과한다. 확신이 없으면 좁히지 말고 넓은 타입 (`object`, 유니온)을 쓴다. 틀린 애너테이션은 없는 것보다 나쁘다.
- 첫 단어의 대문자 표기를 강제하지 않는다(\$8.1에서 `D403` 을 끈다). 도구·명령 이름으로 문장을 시작하는 독스트링에서 자동 수정이 `ffmpeg` 를 `Ffmpeg` 로, `ack` 를 `Ack` 로 바꿔 틀린 문장을 만든다. 소문자가 정확한 표기인 고유명은 그대로 둔다.

6. 주석 원칙

- 주석은 **무엇(what)이 아니라 왜(why)**를 적는다. 코드를 읽으면 아는 내용은 쓰지 않는다.
- 다음의 경우 주석을 필수로 남긴다.
 - 비직관적인 우회 처리 — 외부 API 버그 회피, 하드웨어 제약 대응
 - 성능을 위해 가독성을 희생한 부분
 - 도메인 규칙에서 유래한 상수·임계값 — 근거와 출처
 - 의도적으로 처리하지 않은 예외 케이스
 - 동시성·순서 의존성이 있는 코드
- 임시 코드는 `# TODO(담당자): 내용`, 알려진 결함은 `# FIXME(담당자): 내용` 형식으로 남긴다.
- 주석 처리된 죽은 코드는 커밋하지 않는다. 이력은 git이 관리한다.

7. 규칙별 강제 수단

CLAUDE.md와 규칙 파일은 컨텍스트이치 강제 설정이 아니다. 일관성을 보장하려면 아래와 같이 린터·훅·게이트로 이중화해야 한다.

이 표의 「수준」은 사실이어야 한다. 검사할 수 없는 것을 「완전 자동」으로 적어 두면, 아무도 안 보는 규칙이 지켜지고 있다고 착각하게 된다 — 규칙을 어긴 것보다 나쁘다(\$7.1).

규칙	강제 수단	수준
대소문자·네이밍 형식	ruff <code>N</code> (pep8-naming)	완전 자동
동사_목적어 함수명	<code>tools/check_naming.py</code> (AST 검사)	완전 자동
구글 독스트링 형식·인자 일치	ruff <code>D</code> + <code>convention = "google"</code> , 인자·반환은 <code>pydoclint</code>	완전 자동

Raises 정확성	코드 리뷰 (§5 — 도구는 전파를 못 본다)	사람 판단
타입 힌트 존재	ruff ANN	완전 자동
타입 힌트 정확성	코드 리뷰. 도입하려면 §8.9 의 타입 검사기	사람 판단
변수 이름이 뜻을 담는가	코드 리뷰 (§2 — 도구가 뜻을 못 본다)	사람 판단
디렉터리 구조 (12장)	코드 리뷰. 도입하려면 §8.9 의 구조 검사	사람 판단
중복 제거	pylint R0801 또는 jscpd	근사 검출
함수 길이·복잡도	ruff C90 (mccabe), PLR0915	완전 자동
중요 로직 주석	코드 리뷰 체크리스트	사람 판단

「완전 자동」은 §8.5.1의 트리거를 하나라도 둔 저장소에서만 사실이다. 트리거가 없으면 그 검사는 누군가 기억할 때만 도는 것이므로, 그 저장소에서는 해당 행을 「사람 판단」으로 고쳐 적는다.

7.1. 세 계층의 역할 분담

검사 도구를 언제 돌리느냐에 따라 잡히는 것이 달라진다. 셋은 대체 관계가 아니다.

계층	적용 시점	커버 대상	보장
규칙 파일 (§8.4)	코드를 쓰기 전	Claude Code가 쓰는 코드	확률적
훅 (§8.6)	Claude Code가 파일을 쓴 직후	Claude Code가 방금 쓴 파일 하나	결정적, 파일 단위
게이트 (§8.5)	§8.5.1의 트리거가 부를 때	저장소 전량	결정적, 최종 관문

게이트만이 저장소 전량을 본다. 훅은 Claude Code가 건드린 파일만 보므로 사람이 직접 쓴 코드에는 작동하지 않고, 다른 파일이 깨져도 모르며, 개인이 `disableAllHooks` 로 끌 수도 있다. 검사기 자체가 망가진 경우도 훅은 잡지 못한다 — 그 파일을 건드리지 않으면 돌지 않기 때문이다. 훅과 규칙 파일은 게이트에 도달하기 전에 문제를 줄여 주는 완충재이지 강제의 근거가 아니다.

검사할 수 없는 규칙을 검사되는 것처럼 적지 마라. 규칙을 세울 때는 "이것을 무엇이 확인하는가"를 함께 정하고, 확인할 도구가 없으면 §7 표에 **사람 판단**으로 적는다. 도구가 `_존재 여부`만 보는데 `_내용`이 맞다고 믿으면 위반이 조용히 쌓인다 — 애너테이션은 있는데 타입이 틀린 코드, 디렉터리 규칙은 있는데 아무도 안 보는 배치가 그 결과다. 강제하고 싶으면 §8.9의 선택 검사를 붙이고, 붙이기 전까지는 표를 사실대로 둔다.

검사 결과는 **종료 코드로 판정한다**. 출력 문자열을 세는 방식(`grep -c`)은 도구의 출력 형식이 예상과 다르면 **한 건도 매칭하지 못한 채 0을 돌려주고**, 그것이 "위반 없음"과 구별되지 않는다. 실제로 `pydoclint`는 위반 줄 앞에 줄 번호를 붙여 내보내므로 `'^ *DOC'`로 세면 언제나 0이다. 게이트가 도구를 직접 실행하고 그 종료 코드를 쓰는 이유가 이것이다.

계층마다 도구 버전을 맞춘다. §8.6의 혹은 PATH의 `ruff`를 부르고, §8.3의 pre-commit은 `rev`에 고정된 버전을 쓰며, §8.5의 게이트는 트리거가 설치한 버전을 쓴다. 셋이 다르면 혹은 통과시킨 파일을 pre-commit이 다시 고치거나 게이트가 막는다. 규칙 집합은 `ruff` 버전마다 달라지므로(새 규칙이 추가된다) 버전 차이는 곧 검사 범위 차이다. 도입 시 세 곳의 버전을 한 번에 정하고, 올릴 때도 함께 올린다.

본 문서의 편은 개정 시점의 값이며 시간이 지나면 낡는다. 도입자가 이미 더 새로운 `ruff`를 쓰고 있으면 도입 시점부터 §7.1 위반 상태로 시작한다. §10.2의 12단계가 설치 후 `ruff --version`을 핀과 대조해 불일치를 보고하도록 되어 있고, 편을 올릴 때는 본 문서를 고치고 `RULESET_VERSION`을 올린다(§11).

규칙을 무시 목록에 넣을 때는 같은 사안을 재는 형제 규칙이 있는지 함께 본다. `PLR0913`(인자 수)과 `PLR0917`(그중 위치 인자 수)처럼 한 결정이 두 규칙에 걸쳐 있는 경우가 있다. 하나만 끄면 결정이 반만 반영되고, 도구가 판올림되며 나머지 반이 뒤늦게 터진다. 실제로 `PLR0917`은 `ruff 0.16`에서 기본 활성화가 되어 FastAPI 저장소 전반을 막았다 — 라우트 핸들러의 쿼리 파라미터가 곧 함수 인자라 `PLR0913`을 끈 이유가 그대로 적용되는데도 목록에 없었다.

본 문서가 배포하는 파일도 본 문서의 규칙을 통과해야 한다. 8장의 코드 블록은 `ruff check`, `ruff format --check`, `pydoclint`, `check_naming.py`를 모두 통과한 상태로 유지한다. 표준이 배포한 파일이 표준의 게이트를 깨뜨리면 규칙 전체의 신뢰가 무너진다. 개정할 때는 블록을 실제 파일로 기록해 네 검사를 돌려 보고 반영한다. **다만 이 자체 검사로는 안 잡히는 결함이 있다** — §8.2의 `check_naming.py`에는 인자가 많은 함수가 없어서 `PLR0917` 같은 규칙은 실제 저장소에 붙여야 드러난다.

규칙끼리 충돌하지 않는지 확인한다. 서로 다른 도구가 같은 대상에 반대되는 것을 요구하면 어느 쪽을 골라도 게이트가 실패해 저장소가 영구히 막힌다. 도구를 추가하거나 옵션을 바꿀 때는 실제로 돌려 확인한다. 지금까지 확인된 충돌 네 건은 §8.5의 옵션과 §1의 테스트 제외로 해소되어 있다 — 독스트링 타입 표기(§5 대 `pydoclint` 기본값), `__init__` 독스트링(`ruff D107` 대 `pydoclint`

DOC301), 전파 예외 문서화(\$5 대 pydoclint DOC502), 테스트 타입 힌트(\$8.1 ANN 면제 대 pydoclint DOC106 / DOC107).

도구마다 검사대상 범위가 다르다. 같은 예외 경로라도 ruff·pydoclint·check_naming.py 가 각각 다른 방식으로 파일을 모으므로(워킹트리 스캔 / 인자 경로 / git ls-files), 한 곳만 제외하면 나머지가 그 코드에서 실패한다. 특히 submodule과 가상환경은 우리가 고칠 수 없는 코드라 게이트가 영구히 막힌다(\$1, \$8.1, \$8.5).

틀린 결과를 만드는 자동 수정은 규칙에서 뻔다. 혹은 --fix 는 사람이 검토하기 전에 소스를 바꾸므로, 자동 수정이 의미를 훼손하는 규칙은 커 두는 것 자체가 위험하다(\$5의 D403 사례).

8. 저장소 표준 설정 파일

아래 파일들은 모든 파이썬 저장소에 동일하게 배치한다. Claude Code 부트스트랩 절차(10장)가 이 블록을 그대로 기록한다.

기존 저장소에 적용할 때는 pyproject.toml 의 [tool.ruff] 를 반드시 제거한다. ruff는 ruff.toml 을 pyproject.toml 보다 우선한다. 둘을 함께 두면 ruff.toml 이 조용히 이기고, pyproject.toml 에 적어 둔 기존 예외가 예러 없이 전부 무효가 된다. 설정이 두 곳에 있는 것 자체가 사고다.

절차: ① 기존 [tool.ruff] 의 예외 항목을 사유 주석과 함께 새 ruff.toml 로 옮긴다 → ② pyproject.toml 에서 해당 절을 삭제한다 → ③ ruff check . 결과가 이관 전후로 달라진 지점을 확인한다.

8.1. ruff.toml

```
1 target-version = "py311"
2
3 # ruff 는 줄 길이를 **표시 폭**으로 잰다 - 한중일 문자는 2 로 센다.
4 # 주석·독스트링을 한글로 쓰면 실질 상한이 약 50자다. 짧아 보이는 줄이 E501 로
5 # 걸리는 이유가 이것이며, 값을 올리는 대신 문장을 나누는 쪽을 권한다.
6 line-length = 100
7
8 # 1장의 예외 경로 중 저장소 밖에서 온 코드. 벤더링 디렉터리·서브모듈처럼
9 # 우리가 고치지 않는 것을 여기에 적는다. §8.5 의 pydoclint --exclude 와 같은 기준을
10 # 유지한다 - ruff 만 걸러 두면 게이트의 다른 검사가 그 코드에서 실패한다.
11 #
12 # **서브모듈이 있으면 반드시 적는다.** ruff 는 워킹트리를 스캔하므로 서브모듈 코드를
13 # 그대로 읽는 반면, §8.5 의 check_naming.py 는 git ls-files 기반이라 서브모듈을
14 # 보지 않는다. 두 검사의 대상 범위가 다르므로, 비워 두면 우리가 고칠 수 없는 코드로
15 # 게이트가 영구히 막힌다.
16 #
17 # .venv 등 gitignore 된 경로는 ruff 가 자동 제외하므로 여기 적지 않아도 된다.
18 # pydoclint 는 그렇지 않으므로 §8.5 에는 따로 적어야 한다.
19 #
20 # 마크다운은 파이썬 저장소 규칙의 대상이 아니다. ruff 0.16 부터 .md 안의 파이썬
21 # 코드 블록도 포맷 대상이 되는데, 문서의 예제는 좁은 지면에 맞춰 압축해 둔 것이라
22 # 재포맷하면 읽기 어려워진다.
23 extend-exclude = ["*.md"] # + 서브모듈·벤더링 경로. 예: "libcommon", "vendor"
24
25 [lint]
```

```

26 select = ["E", "F", "W", "I", "N", "D", "UP", "B", "SIM", "C90", "PL", "ANN"]
27 ignore = [
28     "D203",
29     "D213",
30     "PLR0913",
31     # 인자 수를 위치 인자만 따로 다시 재는 규칙(ruff 0.16 에서 기본 활성화). 웹 프레임워크의
32     # 라우트 핸들러는 쿼리 파라미터가 곧 인자라 PLR0913 과 같은 이유로 줄일 수 없다.
33     # 둘은 한 결정의 두 면이므로 하나만 끄면 결정이 반만 반영된다 (7.1).
34     "PLR0917",
35     # 독스트링 첫 단어를 대문자로 바꾼다. 자동 수정이 소문자가 정확한 표기인 고유명을
36     # 망가뜨린다 - ffmpeg → Fffmpeg, mpegts → Mpegts, ack → Ack.
37     # 틀린 문장을 만드는 자동 수정은 규칙을 지키는 것보다 나쁘다 (7장).
38     "D403",
39     # 저장소가 설계상 피할 수 없는 규칙군은 여기에 사유와 함께 적는다 (11장).
40     # 예: 순환참조를 끊는 지연 import 가 광범위하면 "PLC0415".
41     # 수백 곳에 역제 주석을 다는 것은 규칙이 아니라 소음이다.
42 ]
43
44 [lint.pydocstyle]
45 convention = "google"
46
47 [lint.mccabe]
48 max-complexity = 10
49
50 [lint.per-file-ignores]
51 # 패키지·서비스 단위로 테스트를 두는 구조에서도 매칭되도록 `**/` 를 앞에 둔다.
52 # "tests/**/*.py" 로 두면 저장소 루트 바로 아래 tests/ 만 걸려진다.
53 #
54 # 테스트에서 의미가 뒤집히는 규칙만 넣는다.
55 #   PLR2004 assert len(x) == 3 - 기댓값이 곧 매직 넘버다. 상수로 승격하면
56 #   테스트가 자기 자신을 검증하게 되어 검증력이 사라진다.
57 #   PLC0415 픽스처·모킹의 지연 import 는 표준 패턴이다. 모듈 상단으로 올리면
58 #   모킹 시점이 어긋난다.
59 #   N802 · N812 · PLC2401
60 #   동작을 서술하는 것이 테스트 이름의 목적이다(비ASCII 이름 포함).
61 #   E501 · SIM · C90 은 넣지 않는다 - 테스트도 읽기 쉬워야 하고, 그쪽은
62 #   테스트라서 정상인 되는 종류가 아니다.
63 "**/tests/**/*.py" = ["D", "ANN", "PLR2004", "PLC0415", "N802", "N812", "PLC2401"]
64 "**/__init__.py" = ["F401"]
65
66 # 마이그레이션은 생성 도구의 템플릿 형식을 유지해야 하므로 직접 편집을 최소화한다 (1장).
67 # 무시 목록은 사용하는 마이그레이션 도구가 만드는 코드 형식에 맞춰 조정한다.
68 "**/migrations/**/*.py" = ["D", "ANN", "E501", "UP007", "UP035", "I001"]
69
70 # 자동 생성 코드 (1장). 저장소에 없으면 이 줄은 지운다.
71 "**/*_pb2.py" = ["ALL"]

```

8.2. tools/check_naming.py

```

1  """함수·메서드 이름이 {동사}_{목적어} 규칙을 따르는지 검사한다."""
2
3  import ast
4  import sys
5  from pathlib import Path
6
7  # 2.1 의 동사 목록과 항상 일치해야 한다.
8  # 문서와 코드가 갈라지면 사람이 통과시킨 이름을 게이트가 막는다.
9  # fmt: skip 은 압축 배열을 유지하려는 것이다 - 없으면 ruff format 이 한 줄에
10 # 하나씩 펼쳐 이 블록이 ruff format --check 를 통과하지 못한다 (7.1).
11 ALLOWED_VERBS = frozenset(
12     {
13         "get", "set", "create", "build", "make", "load", "save", "read",
14         "write", "parse", "render", "convert", "validate", "check", "run",
15         "execute", "fetch", "send", "update", "delete", "remove", "add",
16         "apply", "compute", "calculate", "extract", "filter", "sort",

```

```

17     "merge", "split", "init", "register", "resolve", "handle",
18     "process", "train", "predict", "infer", "export", "ensure",
19     "main", "test", "list", "find", "iter", "count", "format",
20     "describe", "normalize", "start", "stop", "reset", "refresh",
21     "retry", "rename", "prepare", "drop", "mark", "call", "publish",
22     "subscribe", "request", "poll", "probe", "purge", "assign",
23     "monitor", "manage", "track", "wait", "close", "open", "plan",
24     "scan", "store", "record", "evaluate", "embed", "promote",
25 }
26 ) # fmt: skip
27
28 # 술어 함수는 불리언 변수와 같은 규칙을 따른다 (2장).
29 # 동사를 강제하면 check_is_expired 같은 겹말이 나온다.
30 PREDICATE_PREFIXES = ("is_", "has_", "should_", "can_")
31
32 # 외부 인터페이스가 이름을 정하는 경우 (2장). 프레임워크가 URL·필드명·주입 인자명에서
33 # 이름을 끌어오므로 동사_목적어를 강제하면 드롭인 교체 계약이 깨진다.
34 # pytest 픽스처가 여기 드는 이유는 이름이 곧 주입되는 값이기 때문이다 - `assembled` 를
35 # `build_assembled` 로 바꾸면 그 이름이 테스트 인자로 그대로 들어간다.
36 INTERFACE_DECORATORS = (
37     "router.",
38     "app.",
39     "field_validator",
40     "model_validator",
41     "validator",
42     "pytest.fixture",
43     "pytest_asyncio.fixture",
44     "fixture",
45 )
46
47 # 목록에 없는 동사를 써야 할 때의 예외 표기 (2.1, 11장).
48 # 사유 없는 억제는 리뷰에서 반려한다.
49 SUPPRESS_MARKER = "# naming: allow"
50
51 # {동사}_{목적어} 이므로 밑줄로 나눈 조각이 최소 2개여야 한다.
52 MIN_NAME_PARTS = 2
53
54 # 목적어 없이 동사 하나만으로 뜻이 통하는 관용 이름.
55 BARE_VERB_NAMES = frozenset({"main", "run"})
56
57
58 def _get_decorator_name(node: ast.expr) -> str:
59     """데코레이터 노드에서 이름 문자열을 추출한다.
60
61     Args:
62         node: 데코레이터 표현식 노드.
63
64     Returns:
65         데코레이터 이름. 이름을 뽑을 수 없는 형태면 빈 문자열.
66     """
67     if isinstance(node, ast.Name):
68         return node.id
69     if isinstance(node, ast.Attribute):
70         return node.attr
71     return ""
72
73
74 def _is_exempt(node: ast.FunctionDef | ast.AsyncFunctionDef, line: str, outer_ids: set) -> bool:
75     """이 함수를 네이밍 검사에서 빼야 하는지 판단한다.
76
77     Args:
78         node: 검사 대상 함수 노드.
79         line: 그 함수의 `def` 줄 원문(억제 표기 확인용).
80         outer_ids: 모듈·클래스 몸통에 직접 놓인 함수들의 id 집합.
81
82     Returns:
83         검사에서 제외하면 True.

```

```

84     """
85     # 함수 안에 정의된 함수는 그 함수 밖에서 부를 수 없는 일회용이다(클로저·모킹 대역).
86     # 이름이 계약이 아니라 그 자리의 설명이므로 검사하지 않는다.
87     if id(node) not in outer_ids:
88         return True
89     name = node.name
90     # 던더 메서드와 프로퍼티는 파이썬 관용을 따르므로 검사에서 제외한다.
91     if name.startswith("__") and name.endswith("__"):
92         return True
93     if any(_get_decorator_name(d) in {"property", "cached_property"} for d in node.decorator_list):
94         return True
95     if any(ast.unparse(d).startswith(INTERFACE_DECORATORS) for d in node.decorator_list):
96         return True
97     if name.lstrip("_").startswith(PREDICATE_PREFIXES):
98         return True
99     return SUPPRESS_MARKER in line
100
101
102 def check_file_naming(target_path: Path) -> list[str]:
103     """단일 파일에서 네이밍 규칙 위반 목록을 수집한다.
104
105     Args:
106         target_path: 검사할 파이썬 파일 경로.
107
108     Returns:
109         "경로:줄번호: 메시지" 형식의 위반 문자열 리스트. 위반이 없으면 빈 리스트.
110     """
111     violations: list[str] = []
112     source = target_path.read_text(encoding="utf-8")
113     lines = source.splitlines()
114     tree = ast.parse(source)
115
116     # 클래스 메서드는 클래스명이 목적어 역할을 하므로 동사 하나로 뜻이 통한다 (2장).
117     # ast.walk 는 부모를 알려 주지 않으므로 ClassDef 를 먼저 훑어 메서드를 모은다.
118     method_ids = {
119         id(body_node)
120         for class_node in ast.walk(tree)
121         if isinstance(class_node, ast.ClassDef)
122         for body_node in class_node.body
123         if isinstance(body_node, ast.FunctionDef | ast.AsyncFunctionDef)
124     }
125     module_level = [n for n in tree.body if isinstance(n, ast.FunctionDef | ast.AsyncFunctionDef)]
126     outer_ids = method_ids | {id(n) for n in module_level}
127
128     for node in ast.walk(tree):
129         if not isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
130             continue
131         if _is_exempt(node, lines[node.lineno - 1], outer_ids):
132             continue
133
134         name = node.name
135         parts = name.lstrip("_").split("_")
136         if parts[0] not in ALLOWED_VERBS:
137             violations.append(
138                 f"{target_path}:{node.lineno}: '{name}' 은(는) 허용 동사로 시작해야 한다"
139             )
140         elif (
141             len(parts) < MIN_NAME_PARTS
142             and parts[0] not in BARE_VERB_NAMES
143             and id(node) not in method_ids
144         ):
145             violations.append(f"{target_path}:{node.lineno}: '{name}' 에 목적어가 없다")
146
147     return violations
148
149
150 def main() -> int:

```

```

151     """인자로 받은 파일들을 검사하고 종료 코드를 반환한다.
152
153     Returns:
154         위반이 하나라도 있으면 1, 없으면 0.
155     """
156     all_violations: list[str] = []
157     for raw_path in sys.argv[1:]:
158         all_violations.extend(check_file_naming(Path(raw_path)))
159
160     for line in all_violations:
161         print(line, file=sys.stderr)
162     return 1 if all_violations else 0
163
164
165 if __name__ == "__main__":
166     sys.exit(main())

```

8.3. .pre-commit-config.yaml (선택)

§8.6의 효과 역할이 겹치므로 필수가 아니다. 혹은 Claude Code가 쓴 파일만 보고, pre-commit은 사람이 쓴 코드까지 커밋 시점에 잡는다. 사람이 직접 쓰는 비중이 크면 유지하고, 그렇지 않으면 게이트에만 의존해도 된다. 어느 쪽이든 `pre-commit install` 을 하지 않은 사람에게는 작동하지 않으므로 강제의 근거로 삼지 않는다.

rev 는 §8.6 혹은 부르는 `ruff` 및 §5.1 트리거가 설치하는 `ruff` 와 같은 버전으로 맞춘다 (§71).

```

1 repos:
2   - repo: https://github.com/astral-sh/ruff-pre-commit
3     rev: v0.16.3 # §8.5.2 의 편과 같은 값이어야 한다. 올릴 때는 세 계층을 함께 올린다 (7.1)
4     hooks:
5       - id: ruff
6         args: [--fix]
7       - id: ruff-format
8
9   - repo: local
10     hooks:
11       - id: check-function-naming
12         name: 함수명 동사_목적어 검사
13         entry: python tools/check_naming.py
14         # `system` 이 아니라 `python` 이다. system 은 PATH 에 `python` 이 있다고 가정하는데,
15         # 다수의 리눅스 배포판은 `python3` 만 제공해 혹은 "Executable `python` not found" 로
16         # 실패한다. language: python 이면 pre-commit 이 자체 인터프리터를 준비한다.
17         # 이 검사는 표준 라이브러리만 쓰므로 추가 의존성이 없다.
18         language: python
19         types: [python]
20         # 1장의 예외 경로. ruff 는 per-file-ignores 로 걸려지지만 이 검사는
21         # 스테이징된 파이썬 파일을 그대로 받으므로 여기서 걸러야 한다.
22         # §8.1·§8.5·§8.7 과 같은 기준을 유지한다.
23         exclude: '(/tests/|/migrations/|_pb2\.py$)'

```

8.4. .claude/rules/python-style.md

```

1 ---
2 paths:
3   - "**/*.py"
4 ---
5
6 # 파이썬 코드 작성 규칙

```

```

8 규칙 정보: Confluence 라벨 `python-coding-standard` 문서 (RULESET_VERSION 14)
9 규칙을 바꿔야 하면 이 파일이 아니라 정보 문서를 고치고 `/apply-conventions` 를 재실행한다.
10
11 ## 네이밍
12 - 함수·메서드: {동사}_{목적어} snake_case. 동사는 허용 목록에서 선택.
13 - 목록의 어떤 동사도 그 동작을 정확히 뜻하지 못하면 다른 동사를 쓰되,
14   `# naming: allow <사유>` 로 예외임을 드러낸다.
15 - 변수: 명사 snake_case. 동사로 시작하지 않는다.
16 - 변수 이름은 뜻을 담는다. 타입을 접미로 붙이지 않고(`name_s`), 한 글자 이름은
17   관용이 확립된 자리에만 쓴다. 루프 변수도 무엇을 도는지 담는다 -
18   `for t in turns` 가 아니라 `for turn in turns`.
19 - 추약은 이미 통용되는 것만 쓰고 새로 만들지 않는다(idx.ctx.cfg).
20 - 불리언: is_ / has_ / should_ 접두어.
21 - 클래스: 명사 PascalCase.
22 - 모듈 전역 상수: 대문자 SNAKE_CASE. 그 외 식별자는 소문자.
23 - 내부 전용: 단일 밑줄 접두어.
24 - 클래스 메서드는 클래스명이 목적어 역할을 하므로 동사 하나로 족하다
25   (PROMPTS.get(), STATE.save_relation()). 동사 목록 규칙은 그대로 적용된다.
26 - 술어 함수(is_/has_/should_/can_)와 외부 인터페이스를 따르는 이름은 예외.
27   라우트 핸들러·pydantic 밸리데이터·pytest 픽스처는 데코레이터로 자동 면제된다.
28   프레임워크가 이름으로 읽어 가는 속성(arq 의 redis_settings 등)도 같다 -
29   바꾸면 예외 없이 조용히 기본값으로 떨어진다.
30
31 ## 구조
32 - 같은 로직 3회 반복 또는 함께 수정되어야 하는 2회 반복이면 함수로 추출.
33 - 2개 이상 파일에서 쓰이면 공용 모듈로 분리하고 import.
34 - 함수 50줄 초과, 순환 복잡도 10 초과, 모듈 400줄 초과 시 분해.
35 - 가변 전역 상태와 global 키워드는 금지.
36   예외는 시그널 핸들러의 종료 플래그 하나이며 사유 주석을 남긴다.
37 - 디렉터리 이름은 역할을 말한다. 프레임워크 이름이나 무의미어를 쓰지 않는다(12장).
38
39 ## 독스트링
40 - 구글 스타일. 공개 모듈·클래스·함수·`__init__` 에 필수.
41 - 첫 줄은 명령형 한 문장. Args는 실제 시그니처와 일치.
42 - Args 한 항목이 두 줄을 넘으면 이어지는 줄을 더 깊이 들여쓴다.
43   같은 깊이로 두면 콜론 없는 새 항목으로 읽혀 파싱이 실패한다.
44 - 값을 돌려주면 Returns(제너레이터는 Yields)를 적는다.
45   함수 이름을 되풀이하는 문장은 없는 것만 못하다 - 빈 값·None·단위처럼
46   호출자가 알아야 할 것을 적는다.
47 - Raises 에는 이 함수가 직접 던지는 예외를 적는다. 다만 라우트 핸들러·공개
48   파사드처럼 전파가 곧 계약인 자리에서는 전파되는 예외도 적는다(게이트는 이 항목을
49   검사하지 않으므로 정확성은 네가 책임진다).
50 - 타입 힌트는 실제 타입과 일치해야 한다. 게이트는 존재 여부만 본다 -
51   확신이 없으면 좁히지 말고 넓은 타입을 쓴다. 틀린 애너테이션은 없는 것보다 나쁘다.
52 - 타입은 타입 힌트로 표현하고 독스트링에 중복 기재하지 않는다.
53 - 소문자가 정확한 표기인 고유명(ffmpeg, ack 등)은 문장 첫머리에서도 그대로 둔다.
54
55 ## 주석
56 - 무엇이 아니라 왜를 적는다.
57 - 우회 처리, 성능 트레이드오프, 도메인 유래 상수, 의도적 미처리 예외,
58   동시성·순서 의존성에는 주석을 필수로 남긴다.
59 - 주석 처리된 죽은 코드는 커밋하지 않는다.
60
61 ## 작업 절차
62 - 파일 수정 후 별도 명령을 돌릴 필요는 없다. PostToolUse 혹은 포맷·린트를
63   자동 적용하고, 자동 수정되지 않는 위반은 혹은 되돌려 알려 준다.
64 - 혹은 위반을 알려 오면 그 자리에서 고친다. 역제로 넘기지 않는다.
65 - 혹은 방금 쓴 파일 하나만 본다. 저장소 전량은 `tools/check_all.sh` 가 본다 -
66   커밋 전에 한 번 돌린다.
67 - 검사 결과는 종료 코드로 판정한다. 출력 줄을 세는 방식은 도구의 출력 형식이
68   예상과 다르면 0 을 돌려주고, 그것이 "위반 없음"과 구별되지 않는다.
69 - 새 공용 함수를 만들기 전에 기존 모듈에 같은 기능이 있는지 먼저 검색한다.

```

8.5. tools/check_all.sh — 게이트 정의

저장소 전량을 검사하는 명령을 한 곳에 둔다. 무엇이 이것을 부르는지(§8.5.1)와 분리되어 있어, 트리거를 바꿔도 검사 내용은 그대로다.

검사 명령을 CI 설정 파일 안에만 두지 않는다. 그러면 그 파일을 지우는 순간 "게이트가 무엇인가"의 정의가 함께 사라지고, 남은 사람은 무엇을 돌려야 하는지 알 수 없다.

```
1  #!/usr/bin/env bash
2  # 저장소 전량 게이트. **검사 내용의 유일한 정의**이며, 무엇이 이것을 부르는지(§8.5.1 의
3  # 트리거)와는 분리돼 있다. 트리거가 바뀌어도 검사는 여기 하나로 남는다.
4  #
5  # 1장의 예외 경로는 도구마다 반영 지점이 다르다 - ruff 는 ruff.toml, pydoclint 는 아래
6  # --exclude, check_naming.py 는 아래 grep. 한 곳만 고치면 나머지가 그 코드에서 실패한다.
7  set -uo pipefail
8  cd "$(dirname "$0")/.." || exit 1
9
10 FAILED=""
11
12 step() {
13     printf '
14 — %s
15 ' "$1"
16     shift
17     if ! "$@"; then
18         FAILED="${FAILED} $1"
19     fi
20 }
21
22 step "ruff check" ruff check .
23 step "ruff format" ruff format --check .
24
25 # pydoclint 옵션 네 개는 모두 정본 문서와의 충돌을 막기 위한 것이다.
26 # 하나라도 빠지면 문서가 요구하는 것을 검사가 거부해 저장소가 영구히 막힌다 (7.1).
27 #
28 # --arg-type-hints-in-docstring=False
29 # 5장이 "타입은 타입 힌트로 표현하고 독스트링에 중복 기재하지 않는다" 인데
30 # pydoclint 기본값은 독스트링에 타입을 요구한다.
31 # --check-return-types=False
32 # 같은 이유로 반환 타입도 독스트링에 요구하지 않는다.
33 # --allow-init-docstring=True
34 # 5장과 ruff D107 이 __init__ 독스트링을 요구하는데, pydoclint 기본값은 그것을
35 # 클래스 독스트링에 합치라고 한다(D0C301). 이 옵션이 없으면 붙여도 떼어도 실패한다.
36 # --skip-checking-raises=True
37 # 5장이 "전파가 곧 계약인 경계 함수는 전파되는 예외도 적는다" 인데 pydoclint 는
38 # 몸통의 raise 문만 보므로(D0C502) 그 문서화를 오탐으로 막는다. Raises 정확성은
39 # 리뷰가 본다(§7 표에 "사람 판단"으로 적혀 있다).
40 #
41 # --exclude 는 **ruff 와 대상 범위가 다르다** - ruff 는 gitignore 된 경로를 자동 제외하고
42 # 서브모듈은 ruff.toml 이 걸러 주지만, pydoclint 는 경로를 직접 받으므로 둘 다 걸러지지
43 # 않는다. 가상환경을 빠뜨리면 서드파티 패키지 전체를, 서브모듈을 빠뜨리면 우리가 고칠 수
44 # 없는 코드를 검사한다. 아래 값에 저장소의 서브모듈 경로를 덧붙인다(§8.1 과 같은 기준).
45 # 테스트를 제외하는 이유는 1장에 있다 - §8.1 이 테스트에 D·ANN 을 면제하는데 pydoclint 가
46 # 검사하는 것이 정확히 그 둘이다.
47 step "pydoclint" pydoclint --style=google \
48     --arg-type-hints-in-docstring=False \
49     --check-return-types=False \
50     --allow-init-docstring=True \
51     --skip-checking-raises=True \
52     --exclude '\.git|\.tox|\.venv|venv|\.direnv|node_modules|/tests/' \
53     .
54
55 # 1장의 예외 경로는 여기서도 제외한다. ruff 는 per-file-ignores 로 걸러지지만 이 검사는
56 # 파일 목록을 직접 받으므로 걸러지지 않는다. 서브모듈은 git ls-files 가 gitlink 만 내므로
```

```

57 # 자동으로 빠진다. xargs -r 은 대상이 0건일 때 인자 없이 실행되어 통과한 것처럼 보이는
58 # 것을 막는다.
59 printf '
60 — check_naming
61 '
62 if ! git ls-files '*.py' \
63     | grep -v -E '(/tests/|/migrations/|_pb2\.py$)' \
64     | xargs -r python tools/check_naming.py; then
65     FAILED="${FAILED} check_naming"
66 fi
67
68 # 구조 검사는 §8.9 를 도입한 저장소만 둔다. 없으면 이 블록을 지운다.
69 if [[ -x tools/check_structure.sh ]]; then
70     step "check_structure" ./tools/check_structure.sh
71 fi
72
73 if [[ -n "${FAILED}" ]]; then
74     printf '
75 실패:%s
76 ' "${FAILED}"
77     exit 1
78 fi
79 printf '
80 전부 통과
81 '
82 exit 0

```

8.5.1. 트리거 — 이 게이트를 무엇이 부르는가

트리거	적용 시점	보장
GitHub Actions (§8.5.2)	푸시·PR	결정적. 병합을 막을 수 있다
git pre-push 혹은 (§8.5.3)	푸시 직전	결정적. <code>--no-verify</code> 로 우회 가능
없음 — 사람이 기억해서 부른다	부를 때	확률적

둘 중 하나는 두는 것을 권한다. CI 도구를 쓰지 않기로 한 저장소라면 pre-push 혹은이 그 자리를 대신 한다 — Claude Code 와 무관하게 돌고, 저장소 전략을 보며, 검사기 자체가 망가지면 그 자리에서 드러난다.

트리거를 하나도 두지 않기로 했다면 두 가지를 함께 한다.

- §7 표의 「완전 자동」 행을 그 저장소 기준으로 「사람 판단」 으로 고쳐 적는다. 아무도 부르지 않는 검사를 자동이라고 적어 두면 §7.1 이 경고한 착각이 그대로 생긴다.
- §8.6 혹은 pydoclint 와 구조 검사를 더한다. 그러지 않으면 그 둘은 아무 데서도 돌지 않는다.

8.5.2. .github/workflows/lint.yml

```

1 name: lint
2
3 on: [push, pull_request]
4
5 jobs:

```

```

6 lint:
7   runs-on: ubuntu-latest
8   steps:
9     - uses: actions/checkout@v4
10      # 서버모듈은 체크아웃하지 않는다. 세 검사 모두 서버모듈을 보지 않도록 되어 있고
11      # (ruff.toml extend-exclude · pydoclint --exclude · check_naming 은 git ls-files
12      # 라 gitlink 만 본다), 받아 봤자 우리가 고칠 수 없는 코드다 (1장).
13     - uses: actions/setup-python@v5
14       with:
15         python-version: "3.11"
16      # 버전을 고정한다. §8.3 의 rev, §8.6 혹은 부르는 ruff 와 같은 값이어야 한다 (7.1).
17      # 고정하지 않으면 ruff 가 올라가면서 새 규칙이 들어와 어제 통과한 PR 이 오늘 막힌다.
18     - run: pip install ruff==0.16.3 pydoclint
19     - run: ./tools/check_all.sh

```

8.5.3. .github/hooks/pre-push

git 이 부르는 혹은 원래 `.git/hooks/` 에 있어 커밋되지 않는다. 저장소에 커밋되는 자리에 두고 `core.hooksPath` 로 가리키면 클론한 사람이 한 줄로 켤 수 있다.

```

1 #!/usr/bin/env bash
2 # 푸시 직전 전량 게이트 (§8.5.1 의 트리거).
3 #
4 # 활성화: git config core.hooksPath .githubhooks
5 # 기록 후 chmod +x 를 반드시 실행한다 - 실행 권한이 없으면 git 이 조용히 건너뛴다.
6 exec ./tools/check_all.sh

```

8.6. .claude/settings.json — PostToolUse 혹은

Claude Code가 파이썬 파일을 쓴 직후 포맷·린트를 자동 적용하고, 자동 수정으로 해결되지 않는 위반은 Claude에게 되돌려 준다. `.claude/settings.json` 은 git에 커밋되므로 저장소를 클론한 팀원 전원에게 별도 설치 없이 적용된다.

```

1 {
2   "hooks": {
3     "PostToolUse": [
4       {
5         "matcher": "Edit|Write",
6         "hooks": [
7           {
8             "type": "command",
9             "if": "Edit(**/*.py)",
10            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/check_python_file.sh",
11            "args": [],
12            "timeout": 60,
13            "statusMessage": "파이썬 표준 검사 중..."
14          },
15          {
16            "type": "command",
17            "if": "Write(**/*.py)",
18            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/check_python_file.sh",
19            "args": [],
20            "timeout": 60,
21            "statusMessage": "파이썬 표준 검사 중..."
22          }
23        ]
24      }
25    ]
26  }
27 }

```

설계상 주의할 점:

- `if` 는 규칙을 하나만 담고 한 도구에만 매칭된다. 그래서 `Edit` 과 `Write` 에 각각 핸들러를 둔다. `&&` 나 목록 문법은 없다.
- 경로 자리표시자를 쓸 때는 `args` 를 둔다. `args` 가 있으면 셸을 거치지 않는 `exec` 형식으로 실행되어, 경로에 공백이나 특수문자가 있어도 인용 문제가 생기지 않는다.
- `matcher` 만으로는 부족하다. `matcher`는 도구 이름만 거르므로 `.py` 가 아닌 파일에도 훅이 뜬다. 파일 유형 필터는 `if` 가 담당하고, 스크립트에서 한 번 더 확인한다.
- 훅은 병렬로 실행되며 도구 실행을 되돌리지 못한다. `PostToolUse` 시점에는 파일이 이미 쓰인 뒤다. 훅의 역할은 되돌리기가 아니라 즉시 교정과 피드백이다.
- 훅은 방금 쓴 파일 하나만 본다. 저장소 전량의 상태, 다른 파일의 회귀, 검사기 자체의 고장은 훅의 사각이다 — 그것이 게이트(§8.5)가 따로 있는 이유다.
- 훅은 `pydoclint` 와 구조 검사를 돌리지 않는다. 독스트링과 시그니처의 일치는 자동 수정이 불가능하고, 함수 하나를 고치는 중에 매번 지적하면 작업이 끊긴다. 이 검사는 게이트가 맡는다. 단 §8.5.1의 트리거를 두지 않은 저장소는 훅에 둘을 더한다 — 그러지 않으면 어디에서도 돌지 않는다.

8.7. `.claude/hooks/check_python_file.sh`

기록 후 `chmod +x` 를 반드시 실행한다. 실행 권한이 없으면 훅이 조용히 비활성 상태가 된다 (§8.8).

```
1  #!/usr/bin/env bash
2  # PostToolUse 훅. Claude Code 가 파이썬 파일을 쓴 직후 포맷·린트를 적용하고,
3  # 자동 수정으로 해결되지 않는 위반은 exit 2 로 Claude 에게 되돌려 준다.
4  #
5  # PostToolUse 는 도구를 되돌리지 못한다. exit 2 의 효과는 stderr 를 Claude 에게
6  # 보여 주는 것뿐이며, Claude 는 그것을 읽고 곧바로 파일을 고친다.
7  set -uo pipefail
8
9  # 훅 입력은 stdin 으로 들어오는 JSON 이다. 파일 경로를 담은 환경변수는 없다.
10 INPUT=$(cat)
11 FILE_PATH=$(printf '%s' "${INPUT}" | jq -r '.tool_input.file_path // empty')
12
13 # 대상이 없거나 파이썬이 아니면 조용히 통과한다.
14 if [[ -z "${FILE_PATH}" || "${FILE_PATH}" != *.py || ! -f "${FILE_PATH}" ]]; then
15     exit 0
16 fi
17
18 # 1장의 예외 경로. §8.1 per-file-ignores 및 §8.5 게이트와 같은 기준을 유지한다.
19 # 서브모듈·가상환경 경로가 있으면 여기에도 함께 적는다.
20 case "${FILE_PATH}" in
21     */migrations/*|*_pb2.py) exit 0 ;;
22 esac
23
24 # 자동 수정 단계. 출력은 버린다 — 컨텍스트에 넣을 값이 없다.
25 ruff format "${FILE_PATH}" >/dev/null 2>&1 || true
26 ruff check --fix "${FILE_PATH}" >/dev/null 2>&1 || true
27
28 # 보고 단계. 자동 수정 후에도 남은 것만 Claude 에게 전달한다.
29 # 테스트는 네이밍 검사 대상이 아니다 (1장).
30 case "${FILE_PATH}" in
```

```

31     */tests/*) NAMING_OUTPUT=""; NAMING_STATUS=0 ;;
32 *)
33     NAMING_OUTPUT=$(python3 "${CLAUDE_PROJECT_DIR}/tools/check_naming.py" "${FILE_PATH}")
34     NAMING_STATUS=$?
35     ;;
36 esac
37 RUFF_OUTPUT=$(ruff check --output-format concise "${FILE_PATH}" 2>&1)
38 RUFF_STATUS=$?
39
40 if [[ ${NAMING_STATUS} -ne 0 || ${RUFF_STATUS} -ne 0 ]]; then
41     {
42         [[ -n "${NAMING_OUTPUT}" ]] && printf '%s
43 ' "${NAMING_OUTPUT}"
44         [[ -n "${RUFF_OUTPUT}" ]] && printf '%s
45 ' "${RUFF_OUTPUT}"
46         printf '%s
47 ' "위 위반을 지금 수정할 것. 억제 주석으로 넘기지 말 것."
48     } >&2
49     exit 2
50 fi
51
52 exit 0

```

8.8. 흑 도입 시 확인 사항

- `jq` 가 설치되어 있어야 한다. 흑 입력은 stdin JSON이므로 파싱이 필요하다. macOS는 `brew install jq`, 우분투는 `apt install jq`. 온보딩 문서에 포함한다.
- 실행 권한을 확인한다. 스크립트가 실행 불가능하면 셸이 127로 종료하고, Claude Code는 이를 비차단 오류로 처리해 그대로 진행한다. 첫 실행에서 `Failed with non-blocking status code` 알림이 뜨는지 반드시 확인한다. 이 알림을 놓치면 흑이 꺼진 채로 계속 작업하게 된다.
- `ruff` 가 `PATH`에 있고 버전이 맞는지 확인한다. 흑은 `PATH`의 `ruff` 를 부른다. 없으면 자동 수정이 조용히 건너뛰어지고, 버전이 §8.3-§8.5.2와 다르면 흑이 통과시킨 파일을 pre-commit이나 게이트가 막는다(§7.1). `ruff --version` 으로 §8.5.2의 핀과 대조한다.
- `/hooks` 로 등록 상태를 확인한다. 어느 설정 파일에서 온 흑인지, 명령이 무엇인지 볼 수 있다.
- 대상은 수정된 파일 하나뿐이다. 저장소 전체(`.`)를 검사하면 파일을 쓸 때마다 누적되어 느려진다.
- 개인이 `disableAllHooks` 로 끌 수 있다. 조직 차원에서 강제하려면 관리형 정책 설정에 흑을 두어야 하며, 그 경우 사용자·프로젝트 설정에서 끌 수 없다.

8.9. 선택 검사 — 구조·타입

§7 표에서 사람 판단으로 적힌 두 규칙(디렉터리 구조, 타입 힌트 정확성)을 기계로 올리고 싶은 저장소가 쓴다. 필수가 아니다 — 도입하면 §7 표의 해당 행을 그 저장소 기준으로 「완전 자동」으로 올려 적는다.

존량이 있는 저장소는 §10.5 를 먼저 읽어라. 이 검사들은 켜는 순간 기존 위반을 전부 막는다.

구조 검사 — `tools/check_structure.sh`

```
1 #!/usr/bin/env bash
```

```

2 # §12.1 - 디렉터리 이름은 역할을 말해야 한다.
3 # 프레임워크 이름과 무의미어를 금지한다. 저장소가 BANNED 에 이름을 더할 수 있다.
4 #
5 # git ls-files 를 쓰므로 서브모듈(gitlink)과 gitignore 된 경로는 자동으로 빠진다 -
6 # ruff 의 extend-exclude 와 달리 예외 경로를 따로 적지 않아도 되는 이유다 (7.1).
7 set -uo pipefail
8
9 BANNED='fastapi|django|flask|tornado|starlette|express|nest|functions|misc|stuff|temp|tmp|etc
10
11 BAD=$(git ls-files \
12     | awk -F/ '{ for (i = 1; i < NF; i++) print $i }' \
13     | sort -u \
14     | grep -x -E "${BANNED}" || true)
15
16 if [[ -n "${BAD}" ]]; then
17     printf '§12.1 위반 - 역할을 말하지 않는 디렉터리 이름:
18 '
19     printf '  %s
20 ' "${BAD}"
21     printf '12장의 구조를 참고해 이름을 바꾼다. 이관은 §10.5 의 순서를 따른다.
22 '
23     exit 1
24 fi
25 exit 0

```

`utils` · `common` · `helpers` 는 기본 목록에 넣지 않았다. 무의미어에 가깝지만 관용으로 굳은 이름이라 일괄 금지하면 소음이 크다 — 저장소가 판단해 `BANNED` 에 더한다.

타입 검사기 (선택)

§7 이 적은 대로 `ANN` 은 애너테이션의 존재만 본다. 정확성까지 기계로 보려면 `tools/check_all.sh` 에 아래를 더한다.

```

1 | step "mypy" mypy --ignore-missing-imports --no-strict-optional .

```

존량이 있는 저장소는 관대한 설정에서 시작해 단계적으로 좁힌다. 처음부터 `strict` 로 켜면 §10.5 의 3단계를 건너뛴 것과 같다.

9. Claude Code 적용 방법

9.1. 3계층 배포

계층	위치	범위
조직 전체	관리형 정책 설정 — macOS /Library/Application Support/ClaudeCode/, Linux/WSL /etc/claude- code/, Windows C:\Program Files\ClaudeCode\.	전 구성원 기기. 개인 설정으로 제외 불가

	CLAUDE.md와 혹은 모두 배포 가능	
저장소	<code>.claude/rules/python-style.md</code> , <code>.claude/settings.json</code> , <code>.claude/hooks/</code> (git 커밋)	해당 저장소 작업자 전원
개인	<code>~/.claude/settings.json</code> , <code>~/.claude/CLAUDE.md</code>	본인 전용 보조 설정

혹 항목은 설정 계층 간에 **병합**된다. 개인 설정에 혹은 추가해도 저장소나 관리형 설정의 혹은 사라지지 않는다.

9.2. 신규 프로젝트 초기값

- 사내 템플릿 저장소(`python-template`)에 8장의 파일 일체와 12장의 디렉터리 골격을 포함시키고, 신규 프로젝트는 반드시 이 템플릿에서 생성한다. **신규는 처음부터 맞추는 것이 유일하게 싼 시점이다** — 기존 저장소가 \$10.5 를 거쳐야 하는 이유가 그것이다.
- 여러 저장소가 같은 규칙을 공유해야 하면 규칙 저장소를 하나 두고 심볼릭 링크로 연결한다. `.claude/rules/` 는 심볼릭 링크를 지원한다.
- 비개발자 개발 지원 플랫폼이 프로젝트를 자동 생성하는 경우, 이 템플릿을 시드로 사용한다.

10. Claude Code 자동 적용 절차

Claude Code가 본 문서를 스스로 찾아 읽고 저장소 설정을 구성한다. 전제 조건은 Claude Code에 Atlassian MCP 서버가 연결되어 있을 것.

10.1. 문서 식별 — 페이지 ID를 쓰지 않는 이유

Confluence 페이지 ID는 제목 변경이나 위치 이동에도 유지되지만, 문서를 **복사해 새로 만들거나 삭제 후 재생성**하면 달라진다. 자동화가 ID에 묶여 있으면 그 시점에 조용히 실패한다. 따라서 자동화는 ID가 아니라 **라벨**로 문서를 찾는다.

- 본 문서에는 라벨 `python-coding-standard` 를 부여한다. 이 라벨은 사내 전체에서 본 문서 하나에만 붙인다.
- 문서를 재작성·이관하더라도 **라벨만 새 문서로 옮기면** 모든 저장소의 자동화가 그대로 동작한다.
- 저장소에는 페이지 ID를 저장하지 않는다. 커맨드가 매번 라벨로 조회한다.

10.2. `.claude/commands/apply-conventions.md`

```
1 ---
2 description: 사내 코딩 컨벤션 정보 문서를 찾아 읽고 저장소 설정 파일을 생성·갱신한다
```

```

3 ---
4
5 다음 절차를 순서대로 수행하라. 페이지 ID를 임의로 가정하지 말 것.
6
7 1. Atlassian MCP 의 searchConfluenceUsingCql 로 정본 문서를 찾는다.
8   cql: `label = "python-coding-standard" AND type = page`
9   - 결과가 1건이면 그 문서를 사용한다.
10  - 2건 이상이면 목록을 사용자에게 제시하고 선택을 받는다. 임의 선택 금지.
11  - 0건이면 아래 대체 조회를 시도한다.
12    `title ~ "파이썬 코드 작성 규칙" AND type = page`
13  - 그래도 0건이면 작업을 중단하고, 라벨이 유실되었을 가능성을 사용자에게 보고한다.
14 2. getConfluencePage 로 해당 문서를 markdown 형식으로 읽는다.
15 3. 문서 상단의 RULESET_VERSION 값을 기록해 둔다.
16 4. 문서에 검토 중임을 나타내는 표식이 남아 있으면 기록을 중단하고 보고한다.
17   확정되지 않은 규칙을 저장소에 배포하면 검토 중인 내용이 게이트로 강제된다.
18 5. "8. 저장소 표준 설정 파일" 섹션의 각 코드 블록을 아래 경로에 기록한다.
19   - ruff.toml
20   - tools/check_naming.py
21   - tools/check_all.sh (게이트 정의 - 필수)
22   - .pre-commit-config.yaml (선택 - 8.3 참조. 도입 여부를 사용자에게 묻는다)
23   - .claude/rules/python-style.md
24   - .claude/settings.json
25   - .claude/hooks/check_python_file.sh
26 8.5.1 의 트리거는 **사용자에게 물어 고른다.** GitHub Actions 를 쓰면
27   .github/workflows/lint.yml, git hooks 쓰면 .githubhooks/pre-push 를 기록한다.
28   둘 다 두지 않기로 하면 8.5.1 이 요구하는 두 가지($7 표 수정, 혹은 확장)를 함께 한다.
29   8.9 는 선택이다. 도입 여부를 사용자에게 묻고, 도입하면 tools/check_structure.sh
30   기록과 tools/check_all.sh 반영을 함께 한다.
31   기록한 내용이 문서의 코드 블록과 한 글자도 다르지 않은지 확인한다. 특히 주석은
32   규칙의 근거를 담고 있으므로 빠뜨리면 다음 사람이 이유를 모른 채 규칙을 바꾼다.
33 6. 기존 파일이 있으면 덮어쓰기 전에 diff 를 출력하고 사용자 승인을 받는다.
34   .claude/settings.json 에 이미 다른 훅이 있으면 지우지 말고 병합한다.
35 7. `chmod +x` 를 실행한다(.claude/hooks/check_python_file.sh, tools/check_all.sh,
36   도입했다면 .githubhooks/pre-push · tools/check_structure.sh 도).
37   실행 권한이 없으면 훅이 비차단 오류로 처리되어 조용히 꺼진 상태가 된다.
38   .githubhooks 를 기록했다면 `git config core.hooksPath .githubhooks` 도 실행한다.
39 8. `jq` 가 설치되어 있는지 확인한다. 없으면 설치 방법을 안내하고,
40   훅이 동작하지 않는 상태임을 명시적으로 보고한다.
41 9. 기존 저장소라면 pyproject.toml 에 [tool.ruff] 가 남아 있는지 확인한다.
42   남아 있으면 ruff.toml 이 우선하여 기존 예외가 무효가 되므로,
43   8장 서두의 이관 절차를 사용자에게 안내하고 승인 없이 진행하지 않는다.
44 10. 1장의 예외 경로(벤더링·서브모듈·자동 생성 코드)를 저장소에서 찾아
45   ruff.toml 의 extend-exclude, tools/check_all.sh 의 pydoclint --exclude,
46   8.3 의 pre-commit exclude, 8.7 의 훅 스크립트에 **같은 기준으로** 반영한다.
47   한 곳만 넣으면 나머지 검사가 그 코드에서 실패한다.
48   서브모듈은 `git submodule status` 로, 가상환경은 `.gitignore` 로 찾는다.
49   **게이트를 켜기 전에 이 단계를 반드시 끝낸다** - 서브모듈·가상환경이 남아 있으면
50   우리가 고칠 수 없는 코드로 게이트가 영구히 막힌다.
51 11. `.claude/conventions.lock` 에 동기화 결과를 기록한다.
52   ruleset_version, 동기화 일시, 사용한 페이지 ID와 제목, 고른 트리거,
53   도입한 선택 검사.
54   (기록은 이력 추적용이며, 다음 실행 시 조회 기준으로 쓰지 않는다.)
55 12. 8.5.2 에 고정된 버전으로 ruff 와 pydoclint 를 설치한다.
56   훅·pre-commit·게이트가 같은 ruff 버전을 보게 하기 위한 것이다 (7.1).
57   저장소가 쓰는 패키지 관리자를 먼저 확인한다 - uv 프로젝트의 .venv 에는
58   pip 가 없으므로(uv 가 만든 venv 의 기본값) `pip install` 이 실패한다.
59   uv : pyproject.toml 의 dependency-group 에 적고 `uv sync --group dev`
60   poetry : `poetry add --group dev`
61   그 외 : `pip install ruff==<핀> pydoclint`
62   설치 후 `ruff --version` 이 8.5.2 의 핀과 일치하는지 확인하고, 다르면
63   (PATH 에 다른 ruff 가 먼저 잡히는 경우) 사용자에게 보고한다 - 혹은 PATH 의
64   ruff 를 부르므로 그대로 두면 훅과 게이트가 다른 규칙 집합으로 검사한다.
65   핀이 현행보다 많이 낮았다면 정본 문서 갱신을 사용자에게 제안한다.
66   8.3 을 도입했다면 pre-commit 설치와 `pre-commit install` 도 실행한다.
67 13. 방금 기록한 tools/check_naming.py 자체가 ruff check, ruff format --check,
68   pydoclint, 그리고 자기 자신의 네이밍 검사를 통과하는지 확인한다.
69   통과하지 못하면 정본 문서의 결함이므로 사용자에게 보고한다.

```

```

70 14. 훅이 실제로 등록되었는지 확인한다. 파이썬 파일 하나를 사소하게 수정해
71 훅이 실행되는지 보고, 실행되지 않으면 원인을 보고한다.
72 15. `tools/check_all.sh` 를 실제로 실행하고 검사별 위반 건수를 요약한다.
73 **게이트를 한 번도 돌리지 않은 채 적용을 끝내지 마라** - ruff 만 보고 끝내면
74 pydoclint 위반이 첫 푸시에서 한꺼번에 터진다.
75 **판정은 종료 코드로 한다.** 출력 줄을 세는 방식은 도구의 출력 형식이 예상과
76 다르면 0 을 돌려주고, 그것이 "위반 없음"과 구별되지 않는다 (7.1).
77 위반이 서브모듈·가상환경 경로에서 나오면 10 단계가 빠진 것이므로 되돌아간다.
78 **위반이 전부 무시 목록의 형제 규칙이면**(PLR0913 은 켜는데 PLR0917 이 끄는 식)
79 정보 문서의 결합일 가능성이 높다 - 저장소 설정을 고치기 전에 보고한다(11장).
80 이 시점에 일괄 자동 수정은 하지 않는다. 다만 훅이 켜져 있으므로 이후 Claude 가
81 건드리는 파일은 자동으로 수정된다는 점을 사용자에게 알린다.
82 16. **존량 위반이 남았으면 10.5 의 단계를 따른다.** 트리거를 그대로 켜면
83 모든 푸시가 즉시 막힌다. 어느 단계에 있는지 사용자에게 확인받고 진행한다.
84 17. 기존 저장소라면 12장의 디렉터리 구조와 대조한다. 어긋나면 **즉시 옮기지 말고**
85 이관 계획을 문서로 내어 승인을 받는다 - 디렉터리 재배치는 import 수백 곳과
86 문서를 함께 옮기기므로, 설정 파일 배치와 같은 커밋에 섞이면 diff 를 읽을 수 없다.
87 순서는 참조가 적은 것부터: 하위 디렉터리 이름 → 서버 루트 이름 → 배포 경로.
88 각 단계마다 전체 테스트를 돌리고 되돌릴 수 있게 커밋을 나눈다.
89 18. 기록한 파일 목록, ruleset_version, 고른 트리거, 검사별 미해결 위반 건수,
90 다음 권장 조치를 표로 보고한다.
91
92 문서 내용과 저장소 파일이 다르면 항상 문서를 정본으로 간주한다.
93 임의로 규칙을 추가하거나 완화하지 않는다.

```

10.3. 운영 방식

- 신규 저장소에서 `/apply-conventions` 를 1회 실행하면 설정 파일이 생성된다. 페이지 ID를 입력할 필요가 없다.
- 본 문서가 개정되면 RULESET_VERSION 을 올리고, 각 저장소에서 같은 커맨드를 재실행한다. 규칙 문구가 그대로여도 **8장의 파일 내용이 바뀌면 올린다** — 버전이 그대로면 각 저장소가 낡은 사본을 계속 쓰게 되고, 그 사본이 게이트를 깨뜨려도 재동기화 신호가 가지 않는다.
- 세션마다 원격 문서를 읽게 하지 않는다. 매번 읽으면 컨텍스트를 낭비하고 네트워크 실패에 취약해진다. 문서는 **사람의 정본**, 저장소 파일은 **기계가 강제하는 사본**으로 역할을 나눈다.
- 동기화 누락을 막기 위해 정보 문서의 RULESET_VERSION 과 `.claude/conventions.lock` 값을 주기적으로 비교하고 불일치 시 알린다.

10.4. 라벨 유실 대비

- 라벨은 본 문서의 유일한 식별 수단이므로, 문서 이관 시 **라벨 이동을 이관 체크리스트에 포함한** 다.
- 10.2의 1단계에 제목 기반 대체 조회와 명시적 중단 처리가 포함되어 있어, 라벨이 유실되면 잘못된 문서를 적용하는 대신 실패를 보고한다. 조용한 오작동보다 명시적 실패가 안전하다.

10.5. 기존 저장소 점진 도입

존량 부채가 있는 저장소에 §8.5.1의 트리거를 그대로 켜면 **모든 푸시가 즉시 막힌다**. 부채를 다 갚기 전에는 병합이 안 되고, 갚는 작업도 푸시가 필요하므로 교착이다.

이 상태를 방치하면 팀이 게이트를 끄는 쪽을 택한다. **한 번 꺼진 검사는 다시 켜지지 않는다** — 그래서 도입 순서가 규칙 내용만큼 중요하다.

1단계 — 설정 배치 + 자동 수정 (트리거 없음)

\$8 의 파일을 배치하고 `ruff check --fix` · `ruff format` 만 적용한다. 트리거는 아직 두지 않는다.

- 훅(\$8.6)과 규칙 파일(\$8.4)은 이 단계에서 바로 켜다 — 앞으로 쓰는 코드에만 걸리므로 존량과 무관하다.
- 자동 수정은 커밋을 둘로 나눈다: `ruff format` 만 담은 커밋과 `--fix` 를 담은 커밋. 섞으면 포맷 잡음에 묻혀 의미 변화를 못 본다.
- 테스트 전량을 돌려 회귀 0 을 확인하고 커밋한다.

2단계 — 변경분 한정

새로 쓰는 코드부터 막는다. 존량은 건드리지 않으므로 병합이 계속 돌아간다. 트리거에서 `tools/check_all.sh` 대신 아래를 부른다.

```
1 CHANGED=$(git diff --name-only --diff-filter=ACMR \
2   "origin/${BASE:-main}...HEAD" -- '*.py' \
3   | grep -v -E '(/tests/|/migrations/|_pb2\.py$)' || true)
4 # xargs -r: 파이썬 변경이 없을 때 인자 없이 실행되는 것을 막는다.
5 printf '%s\n' "${CHANGED}" | xargs -r ruff check
6 printf '%s\n' "${CHANGED}" | xargs -r ruff format --check
7 printf '%s\n' "${CHANGED}" | xargs -r python tools/check_naming.py
```

이 단계에 오래 머무르지 마라. 변경분 검사는 존량을 영원히 유예하고, 유예된 존량은 리팩터링 때마다 "이 파일을 건드리면 200건이 뜬다"로 돌아온다. 줄여 갈 대상과 기한을 함께 정한다.

변경분 검사는 저장소를 한 번에 훑는 작업과 맞지 않는다. 표준 도입처럼 전 파일을 건드리는 변경에서는 변경분이 곧 전량이라 유예가 성립하지 않고, 파일 이름만 바꾼 커밋도 그 파일 전체를 검사 대상으로 올린다. 그런 작업은 3단계로 바로 가거나, 그 작업 동안만 트리거를 비차단으로 둔다.

3단계 — 전량

존량이 0 이 되면 트리거가 `tools/check_all.sh` 를 그대로 부른다. 이 시점부터 \$7 표의 「완전 자동」이 사실이 된다.

존량을 줄이는 순서

순서	대상	이유
1	자동 수정 가능분 (<code>ruff --fix</code> , <code>format</code>)	비용 0. 남은 것의 성격이 드러난다
2	기계적 수동분 (줄 길이, 독스트링 인자 누락)	판단이 필요 없고 끝이 보인다

3	규칙 범위 문제	고치기 전에 정본을 의심하라 — 위반이 한 규칙에 몰려 있으면 저장소가 아니라 규칙이 틀렸을 수 있다(11장)
4	타입 힌트	모듈 단위로 나눠 점진. 확신 없는 곳은 넓은 타입(\$5)
5	복잡도·구조	리팩터링. 기한을 따로 잡는다

11. 예외 처리와 개정

- 규칙을 어겨야 하는 경우 해당 라인에 `# noqa: RULE` 와 **사유 주석**을 함께 남긴다. 함수명 규칙은 `# naming: allow <사유>` 를 쓴다. 사유 없는 억제는 리뷰에서 반려한다.
- **설계상 피할 수 없는 규칙군은 저장소 단위 ignore 로 처리한다.** 순환참조를 끊는 지연 import 처럼 한 규칙이 저장소 전반에 걸쳐 정당한 경우, 수백 곳에 억제 주석을 다는 것은 규칙이 아니라 소음이다. §8.1 ignore 에 사유 주석과 함께 적고, 그 판단을 팀에 공유한다. **기준은 건수가 아니라 성격이다** — 규칙이 요구하는 대안이 그 저장소에서 더 나쁜 코드를 만든다면 저장소 단위로 끈다.
- 제네릭 경계는 `Any` 를 쓴다. 맥등 실행기·직렬화 유틸·미들웨어처럼 **타입을 좁히면 목적이 사라지는** 자리가 있다. `object` 로 바꾸면 타입 검사기가 속성 접근을 막고, 구체 타입으로 좁히면 그 계층의 존재 이유가 없어진다. `# noqa: ANN401` 와 사유 주석을 함께 남긴다.
- 프레임워크가 이름으로 읽어 가는 속성은 바꾸지 않는다. `arg` 의 `WorkerSettings.redis_settings`, pydantic 모델의 필드 이름처럼 이름 자체가 계약인 자리가 있다. 바꾸면 예외가 나지 않고 **조용히 기본값으로 떨어지거나 값이 사라진다** — 테스트도 통과한다. 값을 만드는 함수 이름과 그 값을 담는 속성 이름은 별개이며, 규칙은 앞의 것에만 적용된다.
- 규칙 개정은 본 문서를 수정하고 RULESET_VERSION 을 올린 뒤, 각 저장소에서 `/apply-conventions` 를 재실행하는 것으로 완료된다. **버전은 문서 안 두 곳에 있다** — 상단 선언과 §8.4 의 규칙 파일. 한쪽만 올리면 배포되는 사본이 거짓 버전을 선언한다.
- 동사 목록 추가, 임계값 변경 등은 팀 합의 후 본 문서에 반영한다. 저장소 설정 파일만 개별 수정하는 것은 금지한다.
- 도구 버전 핀(§8.3 `rev`, §8.5.2 `ruff==`) 갱신도 규칙 개정으로 취급한다 — 세 계층을 한 번에 올리고 RULESET_VERSION 을 올린다(§7.1).
- **규칙이 정보를 지우게 만들면 규칙을 의심한다.** 규칙을 지켰더니 호출자가 알아야 할 것이 사라지거나, 검사를 통과시키려고 사실과 다른 것을 쓰게 된다면 그 규칙이 그 상황을 안 보고 있는 것이

다. 저장소에서 우회하지 말고 정본에 보고한다 — 다음 도입자가 같은 자리에서 같은 선택을 강요받는다.

12. 표준 디렉터리 구조 (python BE 기준)

1	<repo>/	
2	└ <server_name>/	패키지/서버 루트, 필요시 컨테이너명으로 사용
3	├ ── __init__.py	
4	├ ── main.py	컴포지션 루트 (설정 로드 → 어댑터 생성 → 앱 조립)
5	├ ── config.py	pydantic settings (.env 로드)
6	├ ── schemas/	I/O 계약 (요청·응답·도메인 DTO)(필요시)
7	├ ── api/	
8	├ ── routes/	엔드포인트. 파일 하나가 리소스 하나
9	├ ── deps.py	FastAPI 의존성 주입 헬퍼
10	├ ── app/	
11	├ ── core/	
12	├ ── ── <core>.py	서버 주요 기능
13	├ ── services/	유스케이스. 파일 하나가 시나리오 하나
14	├ ── <module_name>/	
15	├ ── ── <function>.py	서버 기능을 위한 모듈
16	├ ── resources/	코드 파일 외 필요
17	├ ── tests/	테스트 코드 (필요시)
18	├ ── shared/	저장소 공통 코드. 서버가 아니므로 서버 루트 밖 (필요시)
19	├ ── deploy/	
20	├ ── .env	실제 .env 는 커밋하지 않는다
21	├ ── .env.example	실제 .env 는 커밋하지 않는다
22	├ ── Dockerfile	
23	├ ── docker-compose.yml	
24	├ ── volume/	docker mount directory, 데이터 저장 위치 - gitignore
25	├ ── dev/front/	개발용 정적 FE. 배포 대상 아님
26	├ ── scripts/	운영·일회성 스크립트
27	├ ── <submodule>/	조직 공통 라이브러리 (git submodule, 수정 금지)
28		

- 기본적인 권장 구조로, 프로젝트에 따라 세부적인 사항의 예외를 둘 수 있다.
- 큰 틀을 벗어나지 않는 범위에서 필요 시 변형 가능하다.
- 서버가 여럿인 저장소는 `tests/` 를 서버별로 둔다. 각 서버가 같은 이름의 최상위 패키지(`app`) 를 가지므로 저장소 루트에서 한 번에 수집하면 패키지 이름이 충돌해 테스트가 모이지 않는다. 실행도 서버 단위로 나누고, 그 규약을 [CLAUDE.md](#) 에 적는다.
- `shared/` 는 저장소 공통 코드의 자리다. 조직 공통(`<submodule>/`)과 다르다 — 이 저장소의 서버들만 쓰는 코드이며, 서버가 아니므로 서버 디렉터리 안에 두지 않는다. 서버가 하나뿐이면 만들지 않는다.

`app/` 아래 세 계층의 경계 — 이름이 역할을 말하려면 기준이 먼저 있어야 한다.

계층	담는 것	의존
<code>core/</code>	여러 모듈이 공유하는 기반 — 설정 어댑터, 로깅, 에러 규격, 공통 클라이언트. 도메인을 모른다	아무것도 안 쓴다

<code><module_name>/</code>	한 도메인 관심사. 이름이 그 도메인을 말한다(<code>memory/</code> · <code>billing/</code> · <code>storage/</code>)	<code>core/</code> 를 쓴다. 그 반대는 없다
<code>services/</code>	유스케이스. 파일 하나가 시나리오 하나. 모듈들을 엮어 하나의 흐름을 만든다	둘 다 쓴다

어디에 둘지 헛갈리면 *****"이것이 우리 도메인을 아는가"*****를 묻는다. 모르면 `core/` , 하나만 알면 `<module_name>/` , 여러 개를 엮으면 `services/` 다.

12.1. 세 가지 원칙

- 디렉터리 이름은 역할을 말해야 한다. 프레임워크 이름(`fastapi/`)이나 무의미어(`functions/`)를 쓰지 않는다.
- 의존은 한 방향으로만 흐른다. 안쪽 계층은 바깥을 모른다.
- 구조 규칙은 리뷰에서 본다. 기계로 강제하려면 §8.9 의 구조 검사를 붙인다 — 붙이기 전까지는 §7 표에 사람 판단으로 적혀 있고, 그것이 사실이다(§7.1).